

An Experimental Study of Prompt Engineering, Self-Attention, and Causal Masking

Haritha Gottumukkala
Department of Applied Data Science
San José State University, California, USA
DATA 266 — Homework 3
SID4: 1598

Abstract—This report presents two experiments on language models. The first compares six prompt engineering techniques using a shopping arithmetic question and a race-ordering puzzle. Twelve examples were executed through LangChain and Ollama using the same language model. Five final answers were correct. Prompts with calculations helped the arithmetic task, but longer explanations often failed to follow the logic clues. The second experiment builds single-head self-attention from scratch using PyTorch. An unmasked model and a causal model were trained on the required paragraph using next-token prediction. Both models learned their word embeddings and query, key, and value projections. The unmasked model reached 77.78% training accuracy, while the causal model reached 71.11%. The trained heatmaps show attention over all 46 word positions. Every future attention weight in the causal model was zero. These experiments demonstrate prompt behavior and causal masking on small tasks; they do not establish performance on unseen data.

Index Terms—prompt engineering, LangChain, Ollama, self-attention, causal masking, next-token prediction

Personal parameters: SID4 = 1598; SEED = 1598; SLICE = 598; HP_ID = 2; CLS_A = 8; CLS_B = 5.

I. INTRODUCTION

Language models can give different answers to the same question when the instructions change. A prompt may provide only a question, show solved examples, request calculations, or ask the model to compare possible plans. Testing these approaches helps explain both their benefits and their failures.

Attention is another important part of language modeling. It lets a token combine information from other positions in a sequence. In an autoregressive model, future tokens must be blocked because they are not available when the next token is generated.

This assignment investigates these ideas through two experiments. The first compares six prompt engineering techniques on two common tasks. The second implements scaled dot-product attention, trains two small models, and visualizes the effect of causal masking.

The attention implementation follows Section 3.2 of *Attention Is All You Need* [1]. The main questions are whether prompt structure improves the answers and whether the trained causal model prevents access to future positions.

II. TASKS, DATASET, AND EXPERIMENTAL SETUP

A. Prompt Tasks

The same two target questions were used across all six techniques. This kept the comparison consistent.

Task 1: Shopping arithmetic.

I buy 3 notebooks at \$4 each and 2 pens at \$1.50 each. I pay with \$20. How many dollars of change should I receive?

The notebook cost is \$12 and the pen cost is \$3. The correct change is:

$$20 - (3 \times 4 + 2 \times 1.50) = 5. \quad (1)$$

The prompt requested a final line containing `FINAL:` followed by the number.

Task 2: Race ordering.

Ava, Ben, and Cara finish a race in first, second, and third place, with no ties. Ava is not first. Ben finishes before Cara. Cara is not second. Give the order from first to third.

Cara cannot be first because Ben must finish before her. Cara cannot be second either, so Cara must be third. Ava is then second because she cannot be first. The correct order is Ben, Ava, Cara.

The prompt requested `FINAL:` followed by the three comma-separated names.

The expected answers were stored in the evaluation code. They were not supplied as answers to the target questions in the prompts.

B. Attention Dataset

The following text was used exactly as provided:

Neural networks are powerful models for learning representations from data. They consist of layers of interconnected neurons. Attention mechanisms allow models to focus on relevant parts of the input. Transformers rely entirely on attention instead of recurrence. Autoregressive models generate text one token at a time.

Whitespace splitting was used for word-level tokenization. Capitalization was preserved, and punctuation remained attached to each word. For example, `data.` was treated as one word token.

The paragraph contained 46 tokens and 40 unique tokens. Vocabulary IDs were assigned in order of first appearance.

TABLE I
ATTENTION DATASET SUMMARY

Property	Value
Dataset	Required paragraph
Tokenization	Word level, whitespace split
Number of tokens	46
Vocabulary size	40
Next-token prediction pairs	45
Word embedding dimension	64
Full attention matrix	46×46

C. Execution Environment

The experiments were completed in Google Colab. Prompt calls used Ollama’s `qwen2.5:3b` model through LangChain’s `ChatOllama` integration [5].

Ordinary calls used temperature 0, seed 1598, a context limit of 4096 tokens, and an output limit of 384 tokens. The model was warmed up before the recorded experiments.

Tree of Thoughts used temperature 0.7 for candidate-plan generation, seeds 1598, 1599, and 1600, and a 256-token limit for each plan. Reviews and final answers used the ordinary model configuration.

Python, NumPy, and PyTorch were seeded with 1598. The attention models ran on the CPU. The notebook recorded package versions and saved the installed package list.

HP_ID was reported, but this assignment did not provide an HP_ID configuration mapping. The complete paragraph was used, so the slice and class parameters were not needed.

III. PROMPT ENGINEERING METHODOLOGY

Each technique had two separately executed code examples: one for arithmetic and one for logic. All responses were obtained through code, and the notebook saved the prompts and outputs.

A. Zero-Shot

The Zero-Shot prompts contained the target question and the required answer format without solved examples.

The arithmetic prompt asked for the change from \$20. The logic prompt asked for the finishing order using the three clues.

This tested whether the model could solve each task directly.

B. Few-Shot

Few-Shot prompts included two solved examples before the target question.

The arithmetic examples were:

Question: I buy 2 erasers at \$2 each and pay \$10.

What is my change?

FINAL: 6

Question: I buy 3 rulers at \$2 each and pay \$10.

What is my change?

FINAL: 4

The logic examples used different names:

Mia finishes before Noah, and Noah finishes before Zoe.

FINAL: Mia, Noah, Zoe

Max is first, and Ivy finishes before Leo.

FINAL: Max, Ivy, Leo

The target question followed these examples. The examples showed the expected output format but did not include worked explanations.

C. Chain-of-Thought

Chain-of-Thought prompts supplied worked examples before the target question [2].

For arithmetic, the examples showed:

Cost = $2 \times 2 = 4$.

Change = $10 - 4 = 6$.

FINAL: 6

A second example showed the ruler calculation. The target question then requested a short calculation.

For logic, the examples explained how the clues placed each person. The target prompt requested a short explanation and a check of the clues.

D. Zero-Shot Chain-of-Thought

Zero-Shot CoT used no solved examples. Instead, it added the instruction “Let’s think step by step” [3].

The arithmetic example also requested a short calculation. The logic example requested a brief check of each clue.

This tested whether asking for an explanation helped without providing demonstrations.

E. Meta-Prompting

Meta-Prompting used two calls per task.

For arithmetic, the first call was:

Write a short, reusable instruction template for solving shopping arithmetic problems. Include how to organize the information and check the answer. Return only the instructions. Do not solve an example.

The generated instructions were then combined with the shopping question in a second call.

For logic, the first call requested a reusable template for ordering puzzles. The second call used that template to solve the race question.

This tested whether model-generated instructions improved the target answer.

F. Tree of Thoughts

A small branching workflow was implemented based on generating and reviewing candidate solutions [4].

For both target tasks, three candidate plans were requested using these approaches:

- 1) Break the task into smaller parts.
- 2) Work backward from the information given.
- 3) Consider possible answers and how to check them.

Each plan was reviewed by the model and assigned a score from 0 to 10. The highest-scoring plan was selected. When scores tied, the first plan was selected.

A final call solved the question using the chosen plan. Each task therefore used seven calls: three plans, three reviews, and one final answer.

This implementation explored one level of candidate plans. It did not perform a deep search with repeated expansion and backtracking.

G. Measurement Procedure

The notebook recorded final-answer correctness, response word count, number of calls, output-token usage, and elapsed time.

The answer checker extracted the last `FINAL:` entry. Arithmetic answers were compared numerically with 5. Logic answers were compared with `Ben`, `Ava`, `Cara`.

The final-answer check did not grade the explanation. Explanations were inspected separately to identify contradictions and ignored clues.

IV. PROMPT RESULTS AND DISCUSSION

A. Observed Answers

Table II shows the outputs across all six techniques. B, A, and C represent Ben, Ava, and Cara.

TABLE II
FINAL-ANSWER COMPARISON

Technique	Math	Logic order	Passed
Zero-Shot	12 <code>FINAL</code>	B, C, A	0/2
Few-Shot	3	B, A, C	1/2
Chain-of-Thought	5	No final answer	1/2
Zero-Shot CoT	5	C, B, A	1/2
Meta-Prompting	5	B, C, A	1/2
Tree of Thoughts	5	B, C, A	1/2

Five of the twelve final answers passed. Arithmetic passed in four of six techniques, while logic passed in only one.

The Chain-of-Thought arithmetic response correctly calculated the notebook cost as \$12, the pen cost as \$3, the total as \$15, and the change as \$5.

The Few-Shot logic response returned the correct order directly. Several longer logic responses returned an order that contradicted a clue.

B. Response Length

The final-response word counts varied substantially.

TABLE III
FINAL-RESPONSE WORD COUNTS

Technique	Math words	Logic words
Zero-Shot	2	160
Few-Shot	2	4
Chain-of-Thought	35	297
Zero-Shot CoT	31	59
Meta-Prompting	75	241
Tree of Thoughts	56	136

These counts cover the final response, including its final-answer text. For Meta-Prompting and Tree of Thoughts, they do not include the intermediate templates, plans, or reviews.

The correct Few-Shot logic answer contained only four words by the notebook’s whitespace-based count. The much longer Chain-of-Thought logic response failed to reach a final answer.

C. Calls, Tokens, and Time

Table IV reports totals across the arithmetic and logic tasks. Token totals include intermediate calls.

TABLE IV
TOTAL PROMPT COST ACROSS BOTH TASKS

Technique	Calls	Output tokens	Time (s)
Zero-Shot	2	233	3.10
Few-Shot	2	13	0.42
Chain-of-Thought	2	445	5.96
Zero-Shot CoT	2	145	2.14
Meta-Prompting	4	1,026	14.28
Tree of Thoughts	14	2,749	45.42

Few-Shot used the fewest output tokens and had the shortest total elapsed time. Tree of Thoughts used the most calls, tokens, and time.

The measurements came from one recorded run after warm-up. They describe this execution and should not be treated as repeated timing benchmarks.

D. Main Findings

Worked calculations and structured instructions helped the arithmetic question. Chain-of-Thought, Zero-Shot CoT, Meta-Prompting, and Tree of Thoughts all produced the correct change.

The logic task showed a different pattern. Few-Shot was the only technique that returned the correct order. The other techniques often restated the clues but failed to enforce them.

More calls did not increase the number of correct answers in this experiment. Meta-Prompting and Tree of Thoughts each passed one task, matching several simpler approaches while using more resources.

The comparison is limited to two questions and one model. It does not show that one technique is always better.

V. SELF-ATTENTION IMPLEMENTATION AND TRAINING

A. Model Workflow

The attention experiment followed this sequence:

- 1) Split the required paragraph into word tokens.
- 2) Build a vocabulary and convert words to IDs.
- 3) Create trainable word and position embeddings.
- 4) Calculate query, key, and value projections.
- 5) Compute scaled dot-product attention.
- 6) Predict the next word and update the parameters.
- 7) Recompute the full attention matrix after training.
- 8) Save and visualize the trained weights.

The causal model used the same workflow with a mask added before softmax.

B. Trainable Embeddings

The word embedding layer used `nn.Embedding(40, 64)`. Its trainable weight table had shape 40×64 .

Looking up all 46 token IDs produced a 46×64 embedding matrix. Trainable position embeddings were added to distinguish word positions.

This was useful for repeated words such as `models` and `of`, which appeared in different contexts and positions.

C. Scaled Dot-Product Attention

Three separate linear layers produced:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (2)$$

where X contains the word and position embeddings.

The attention calculation was:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right), \quad H = AV, \quad (3)$$

with $d_k = 64$.

The core code was:

```
q = self.query(x)
k = self.key(x)
v = self.value(x)

scores = (q @ k.T) / (
    self.embedding_dim ** 0.5
)

attention_weights = torch.softmax(
    scores, dim=-1
)

context = attention_weights @ v
```

A final linear layer converted each context vector into scores for the 40 vocabulary tokens.

The model used basic PyTorch layers, matrix multiplication, softmax, and cross-entropy loss. It did not use `nn.MultiheadAttention`, `nn.Transformer`, `HuggingFace Transformer` classes, or a ready-made attention layer.

D. Next-Token Objective

Each position was trained to predict the next word. The first pair was `Neural` $\rightarrow$ `networks`.

There were 45 prediction targets. The final word had no next-word target, so its loss was ignored.

```
targets = torch.full_like(token_ids, -100)
targets[:-1] = token_ids[1:]

loss_function = nn.CrossEntropyLoss(
    ignore_index=-100
)
```

The loss was the mean cross-entropy over the 45 valid targets. Backpropagation updated the embeddings, projections, and output layer.

E. Training Configuration

The models started from identical parameter values, which were checked before training. Each model then had its own training run using the same settings.

TABLE V
SHARED ATTENTION MODEL CONFIGURATION

Setting	Value
Embedding dimension	64
Attention heads	1
Position embeddings	Trainable
Optimizer	Adam
Learning rate	0.01
Epochs	400 per model
Loss	Cross-entropy
Device	CPU
Seed	1598

F. Causal Masking

The causal model extended the unmasked implementation with a lower-triangular mask.

For query position i and key position j :

$$S_{ij}^{\text{causal}} = \begin{cases} S_{ij}, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (4)$$

Softmax was applied to the masked scores. Therefore, future positions received zero attention weight.

```
causal_mask = torch.tril(
    torch.ones(
        scores.shape,
        dtype=torch.bool,
        device=scores.device
    )
)

masked_scores = scores.masked_fill(
    ~causal_mask, float("-inf")
)

attention_weights = torch.softmax(
    masked_scores, dim=-1
)

context = attention_weights @ v
```

Each token could attend to itself and earlier tokens. The masked model was trained with this restriction throughout training.

VI. ATTENTION RESULTS AND VISUALIZATION

A. Training Results

Table VI reports the final checkpoint results.

TABLE VI
ATTENTION TRAINING RESULTS

Model	Initial loss	Final loss	Accuracy	Time (s)
Unmasked	3.6906	0.6325	77.78%	0.93
Causal	3.7071	0.4621	71.11%	0.92

The unmasked model predicted 35 of 45 training targets correctly. The causal model predicted 32 of 45 correctly.

Both models finished with lower losses than their initial losses. Their initial losses differed because the causal mask

changed the forward calculation, even though their initial parameters matched.

The causal model had lower loss but lower accuracy. Cross-entropy measures the probability assigned to the correct target, while accuracy checks only the highest-scoring prediction. These measures can therefore rank models differently.

B. Training Curves

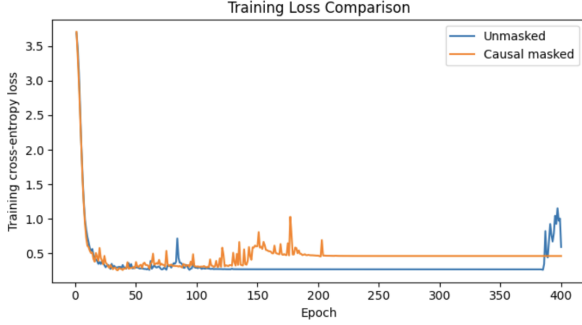


Fig. 1. Cross-entropy training loss for the unmasked and causal models over 400 epochs.

Figure 1 shows that the losses decreased from their starting values, but training was not fully stable.

The unmasked loss was about 0.2698 at epoch 300 and rose near the end. The final evaluation loss was 0.6325.

The epoch 400 training log reported approximately 0.5927 before the last optimizer update. The final loss was evaluated after that update, so the two values differ.

The causal loss also fluctuated and finished near 0.4621. The reported results use the final models rather than an earlier lower-loss checkpoint.

C. Evidence of Learned Parameters

The notebook measured the norm of the difference between the trained and initial weights.

TABLE VII
WEIGHT CHANGES FROM INITIALIZATION

Parameter group	Unmasked	Causal
Word embeddings	7.6681	8.8731
Query projection	8.5001	10.4768
Key projection	7.9213	10.0603
Value projection	7.3406	7.4883

The word embeddings and all three required projections changed in both models. The attention matrices were recomputed after training in evaluation mode.

These checks show that the heatmaps reflect trained parameters rather than the random starting weights.

D. Unmasked Heatmap

Figure 2 shows attention across the complete 46-token sequence. Rows are query positions, and columns are the positions receiving attention. Numbered labels distinguish repeated words.

The unmasked heatmap contains weights on both sides of the diagonal. Some strong links pointed to future words.

For example, *Neural* at position 1 attended strongly to *for* at position 6. The token *Transformers* at position 30 attended strongly to *generate* at position 40.

The maximum future attention weight was approximately 1.0. The mean attention mass assigned to future positions was 0.5428, averaged over all 46 rows.

This confirms that the unmasked model used information beyond the current position.

E. Causal Heatmap

Figure 3 shows the full attention matrix from the separately trained causal model.

The first word could attend only to itself. Later words could attend to more allowed positions, but none could attend to a future word.

The largest future attention weight was exactly 0.0. The mean future attention mass was also 0.0.

The causal heatmap was computed from a model trained with masking. It was not made by covering part of the unmasked heatmap.

VII. TESTING AND VALIDATION

The notebook checked the implementation numerically and through a change to the input.

TABLE VIII
VALIDATION CHECKS AND OBSERVATIONS

Check	Observation
Dataset token count	46 word tokens
Vocabulary count	40 unique tokens
Embedding lookup shape	46×64
Prediction score shape	46×40
Full attention shape	46×46 for both models
Finite attention entries	Passed for both models
Attention row sums	Each row summed to 1 within tolerance
Embedding and Q/K/V updates	Nonzero changes in both models
Causal future weights	Every future weight was zero
Future-token change	Earlier causal logits did not change

For the behavioral check, only the final token ID was changed. Earlier logits from the causal model were then compared with their original values.

The maximum earlier-logit difference was 0.0. This confirms that a change at the final position did not influence predictions at earlier positions in that forward pass.

The trained attention matrices, metrics, parameter changes, and checkpoints were saved by the notebook. The original exported plots are used in this report.

VIII. ERROR ANALYSIS, LIMITATIONS, AND FUTURE WORK

A. Prompt Failures

The Zero-Shot arithmetic response was 12 FINAL. The value 12 was the notebook cost rather than the change. The response also failed to follow the requested final-answer format.

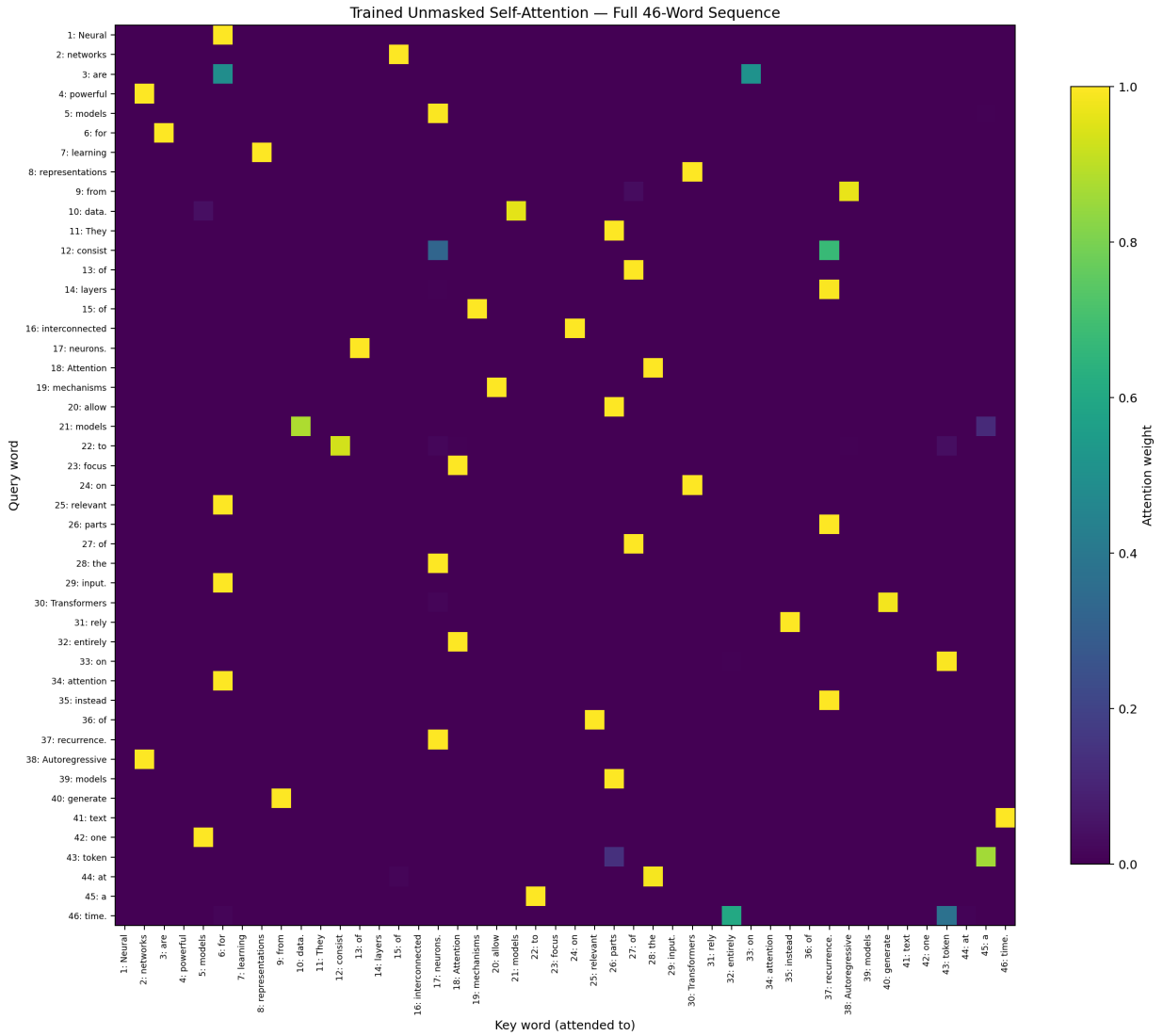


Fig. 2. Learned unmasked self-attention over the complete paragraph. Nonzero weights can appear above the diagonal because future words are available to each query.

Few-Shot arithmetic returned **FINAL: 3**. Since the response did not show its calculation, the precise arithmetic mistake could not be identified.

Several logic responses returned **Ben, Cara, Ava**, which places Cara second and violates a clue. Zero-Shot CoT returned **Cara, Ben, Ava**, which places Cara before Ben.

The Meta-Prompting response explicitly treated “Cara is not second (Cara is second)” as a satisfied clue. This was a contradiction inside the explanation.

The Chain-of-Thought logic response reached the 384-token limit and ended without a final answer. It also contained wrong deductions before truncation.

In Tree of Thoughts, the logic reviewers gave all three invalid plans a score of 10. Selecting a plan by these scores therefore did not protect against a wrong answer.

A future improvement would check candidate race orders

against every clue using explicit constraints. This was not implemented in the reported experiments.

B. Unmasked Information Leakage

The unmasked model could attend to the next word while being trained to predict it. Its higher training accuracy cannot be interpreted as better causal language generation.

Its role in this experiment was to show the behavior of attention without a causal restriction.

C. Training Stability and Fit

Neither model achieved perfect training accuracy. The unmasked model made 10 errors, and the causal model made 13 among the 45 prediction targets.

The late increase in unmasked loss also showed sensitivity to the chosen training settings. A smaller learning rate could be tested in a future run to reduce these fluctuations.

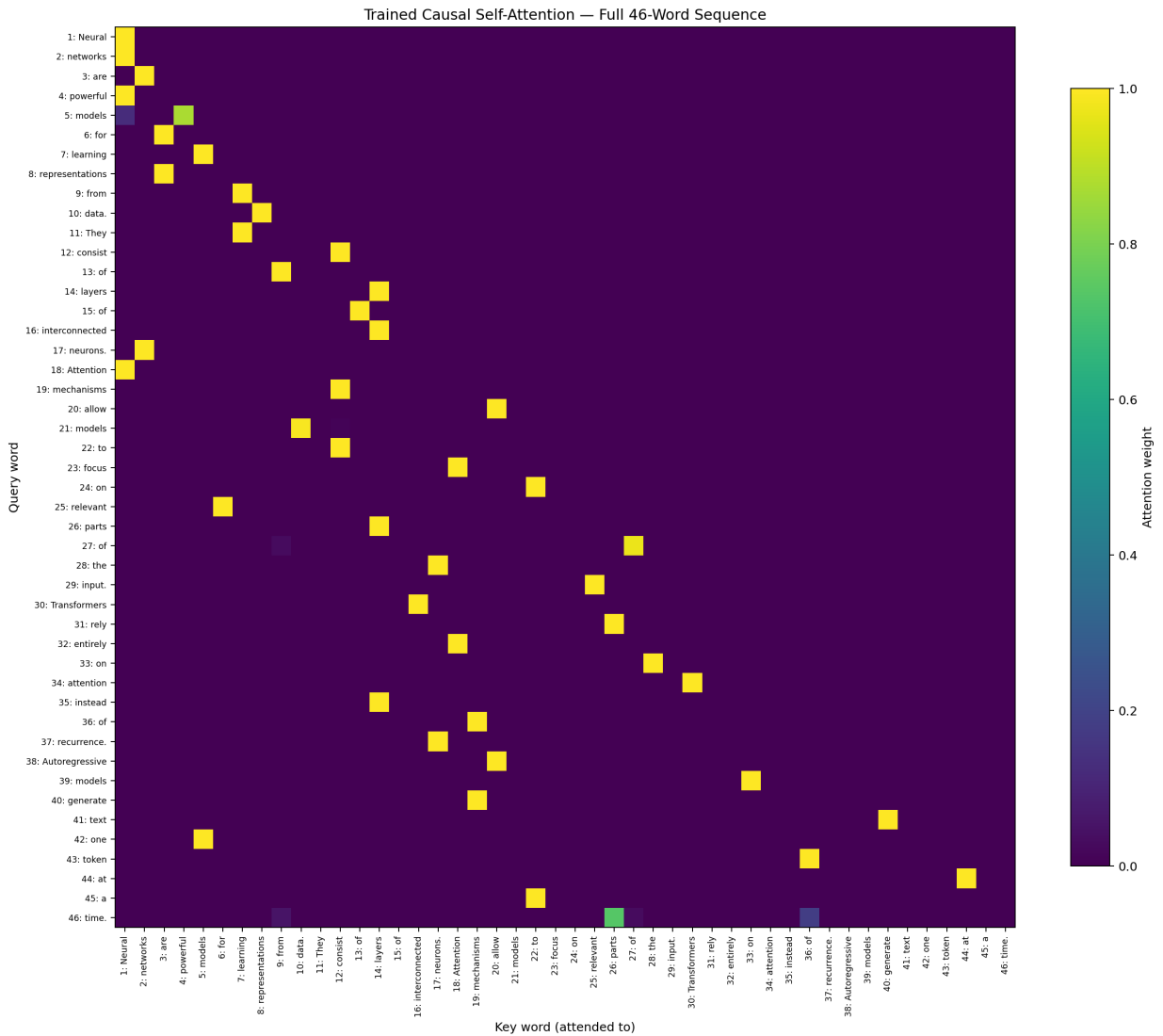


Fig. 3. Learned causal self-attention over the complete paragraph. Every weight above the diagonal is zero. Each query can attend only to its own position and earlier positions.

No additional learning-rate experiment was performed, so no improvement is claimed.

D. Dataset and Evaluation Limits

The dataset was a single short paragraph. Training and evaluation used the same sequence, with no held-out test set.

Trainable position embeddings could help memorize word-position patterns. The reported accuracy therefore measures training fit rather than generalization.

The final word had no training target, although its attention row was included in the required full-sequence heatmap.

E. Limits of Heatmap Interpretation

A strong attention weight identifies where a query gathers information. It does not by itself prove grammatical or semantic understanding.

The heatmaps support a clear structural conclusion: future positions are available in unmasked attention and blocked in causal attention.

Larger datasets and held-out evaluation would be needed to investigate language-model quality beyond this demonstration.

IX. CONCLUSION

This assignment compared six prompt engineering techniques and implemented trained self-attention from scratch.

The prompt experiment produced five correct final answers out of twelve. Calculations and structured instructions helped the shopping task, while only Few-Shot solved the race puzzle correctly. Longer explanations and extra calls did not guarantee correct reasoning.

The attention experiment trained two models using the same paragraph and starting weights. Their word embeddings and query, key, and value projections all changed during training.

The full unmasked heatmap showed access to future words. The causal heatmap contained zero weight at every future position, and changing the last token did not affect earlier causal predictions.

The results demonstrate learned attention and the effect of masking. They also show why training accuracy, confident explanations, and visually strong attention patterns must be interpreted within the limits of the experiment.

REFERENCES

- [1] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [3] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large Language Models are Zero-Shot Reasoners,” in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.11916>
- [4] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.10601>
- [5] LangChain, “ChatOllama integration.” [Online]. Available: <https://docs.langchain.com/oss/python/integrations/chat/ollama>